

4.1 Introduction to Differentiation

```
In [1]: using CairoMakie
using ForwardDiff
using Zygote
using Random
using Distributions
```

In the last few lectures, it was shown that in order to find the minimum of a function, we need to find the derivatives of a function. There are a few methods of finding a derivatives of a function

- Manual
- Symbolic
- Numerical
- Automatic

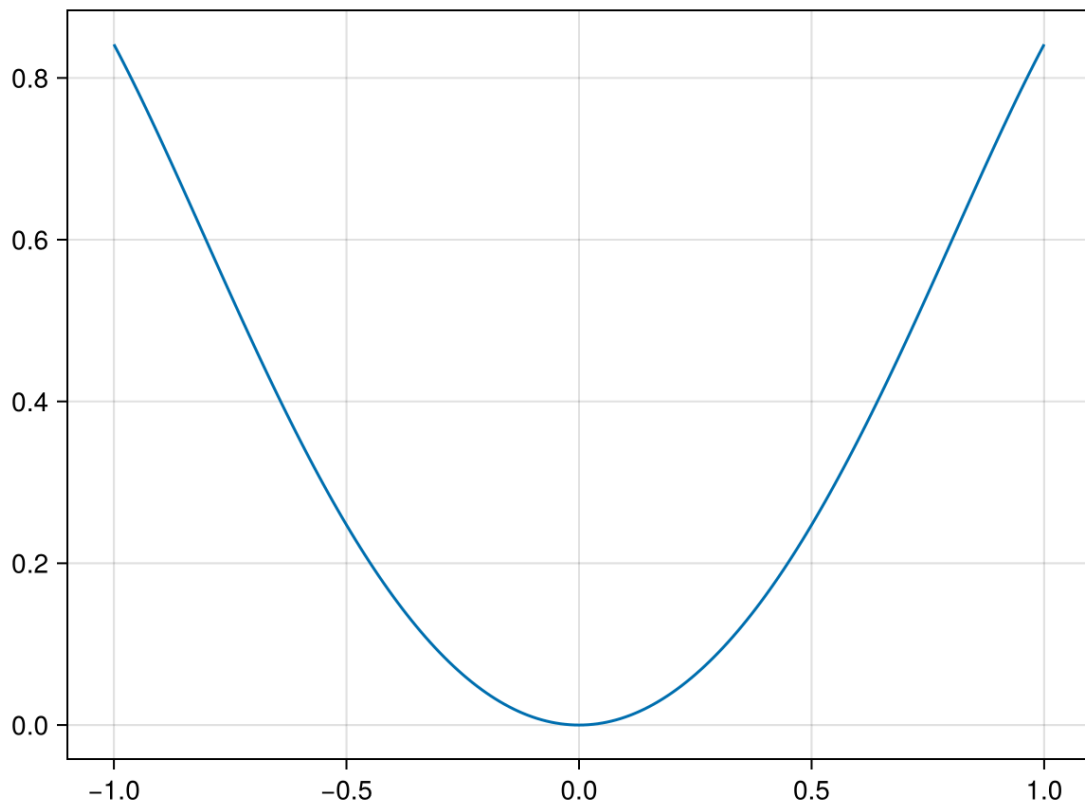
Manual differentiation is just what we learn in high school. Symbolic differentiation is what computer can be programmed to do what we learn in high school. Numerical differentiation is the finite difference approximation of a derivative. Automatic differentiation is not usually talked about much but it is a development that has made many of the machine learning algorithms possible. I will show you each of the different methods and how automatic differentiation is done in Julia.

Consider the function

$$f(x) = \sin(x^2)$$

Let's define and plot it to see how it looks like

```
In [2]: f(x)=sin(x^2);
xplot=-1:0.01:1
lines(xplot,f.(xplot))
```



Manual differentiation is we apply the rules differentiation we learn in high school to get the derivative

$$\frac{df}{dx}(x) = 2x \cos(x^2)$$

To perform Symbolic differentiation in Julia, first we need to import the Symoblics package

In [3]: `using Symbolics`

In [4]: `# Define x as a symbolic variable
@variables x

#Define the function. Note that we cannot define f here because f has be
f1 = sin(x^2)

#Define a differential operator with respect to x
D=Differential(x)

#Expand the expression
expand_derivatives(D(f1))`

`2x*cos(x^2)`

Exercise Ex01: Change code above to calculate the symbolic derivative $\sin(x^2) + \exp(-\cos(x))$.

Symbolic differentiation is feasible if you have a function that is not very complicated and also has only a few variables. When you have a function of many variables (for

real world problems, you can have millions or billions of variables/parameters), then computing symbolic derivatives can get very complicated and might not be possible.

Another way of computing derivatives is using numerical approximation. You should know from other courses/subjects that the derivative of f at x_i can be approximated as

- Forward differentiation: $\frac{df}{dx}(x_i) \approx (f(x_i + h) - f(x_i))/h$
- Backward differentiation: $\frac{df}{dx}(x_i) \approx (f(x_i) - f(x_i - h))/h$
- Central differentiation: $\frac{df}{dx}(x_i) \approx (f(x_i + h) - f(x_i - h))/(2h)$

where h is a small number.

There is another way of numerically approximating the derivatives of a function which might not be commonly known. This is the complex step method. Consider Taylor expansion with an imaginary step ih where i is the imaginary unit.

$$f(x + ih) = f(x) + ih \frac{df}{dx} - \frac{h^2}{2!} \frac{d^2f}{dx^2} - \frac{ih^3}{3!} \frac{d^3f}{dx^3} + \dots$$

Consider now the Real and Imaginary parts of $f(x + ih)$

$$\Re(f(x + ih)) = f(x) - \frac{h^2}{2!} \frac{d^2f}{dx^2} + \dots$$

$$\Im(f(x + ih)) = h \frac{df}{dx} - \frac{h^3}{3!} \frac{d^3f}{dx^3} + \dots$$

$$\frac{1}{h} \Im(f(x + ih)) = \frac{df}{dx} - \frac{h^2}{3!} \frac{d^3f}{dx^3} + \dots$$

From the equations above, it is clear that the the real part of $f(x + ih)$ can approximate $f(x)$ and the derivative of f , df/dx can be approximated by $\frac{1}{h} \Im(f(x + ih))$. The error term is $O(h^2)$.

The code segment below implements the forward, central and complex step methods to calculate the derivative for the function

$$f(x) = \sin(x^2)$$

and compare with the exact value of the derivative

$$df(x)/dx = 2x \cos(x^2)$$

The code below compares at calculates the derivatives at $x = \pi/2$

```
In [5]: dfdx(x)=2*x*cos(x^2)
h=1e-3
xi=pi/2.0
println("forward_diff=$( (f(xi+h)-f(xi))/h )")
println("backward_diff=$( (f(xi)-f(xi-h))/h )")
println("central_diff=$( (f(xi+h)-f(xi-h))/(2*h) )")
println("complex_step=$( imag(f(xi+im*h))/h )")
println("exact_diff=$( dfdx(xi) )")
```

```

forward_diff=-2.4581093000713983
backward_diff=-2.4503856304469718
central_diff=-2.454247465259185
complex_step=-2.4542516170381785
exact_diff=-2.4542495411512917

```

You can see from the numbers that they produce, the central and complex step methods produce comparable accuracy.

To get better accuracy for all methods, you can reduce the value of h .

The code below computes the numerical approximation of the derivatives of $f(x) = \sin(x^2)$ using the forward, backward, central and complex step method. Note that this code is taken and adapted from "Algorithms for Optimization" by Kochenderfer & Wheeler. You can download this book from <https://algorithmsbook.com/>

```

In [6]: #Algorithms for forward, backward, central and complex step method for co
diff_forward(f, x; h=sqrt(eps(Float64))) = (f(x+h) - f(x))/h
diff_central(f, x; h=cbrt(eps(Float64))) = (f(x+h/2) - f(x-h/2))/h
diff_backward(f, x; h=sqrt(eps(Float64))) = (f(x) - f(x-h))/h
diff_complex(f, x; h=1e-20) = imag(f(x + h*im)) / h

#Compute the derivative at x=π
x = π/2
d_true = π*cos(π^2/4) #true value of the derivative

#Compute the derivatives using numerical approximation for h=10^(-17) to
arr_h = collect(10.^ range(-17, stop=1, length=101))
arr_forward = [abs(d_true - diff_forward(f, x, h=h))
               for h in arr_h];
arr_central = [abs(d_true - diff_central(f, x, h=h))
               for h in arr_h];
arr_complex = [abs(d_true - diff_complex(f, x, h=h))
               for h in arr_h];

```

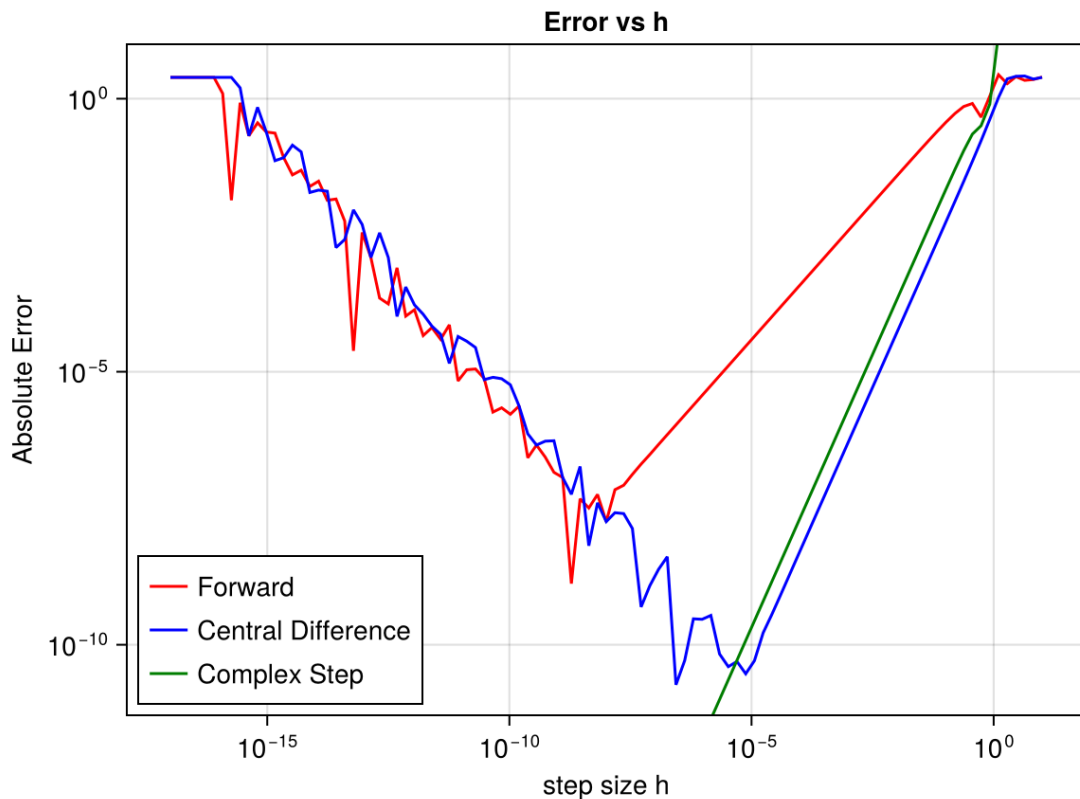
Plotting the error of the approximation for range of h values.

```

In [7]: fig1 = Figure()
ax1 = Axis(fig1[1, 1], xscale=log10, yscale=log10, xlabel="step size h", ylab
lines!(ax1, arr_h, arr_forward; color=:red, label="Forward")
lines!(ax1, arr_h, arr_central; color=:blue, label="Central Difference")
lines!(ax1, arr_h, arr_complex; color=:green, label="Complex Step")
axislegend( position = :lb)

fig1

```



From the graph above, it is clear that you can get Round Off error if h is too small. Hence, for complex functions and for functions with lots of variables, it is not feasible to use numerical differentiation to calculate the derivatives because the truncation and round off errors will usually be too large.

In machine learning, most the derivatives is calculated using a technique called Automatic Differentiation. In automatic differentiation, we can calculate the derivatives to machine precision very quickly and efficiently for a function with many variables. We will go into details of what Automatic differentiation is and how it can be implemented in the next notebook. Here, I will introduce you to some Julia packages that can perform automatic differentiation.

Univariate Function without any parameters

Julia has a few libraries that will compute the derivatives, gradient and Hessians of a function for you. The two packages that will be covered here are called `ForwardDiff.jl` and `Zygote.jl`. They all have slightly different syntax and advantages and disadvantages. `ForwardDiff.jl` is probably the most straightforward to use and `Zygote.jl` is more difficult to use but it is also much faster, especially for machine learning. I will give you examples below on how to use these packages

Using `ForwardDiff.jl` to find the derivative of $f(x)$ at $x = \pi/2$

```
In [8]: ForwardDiff.derivative(f, π/2)
println("The true answer is ",d_true," ForwardDiff gives ",ForwardDiff.de
```

The true answer is -2.4542495411512917 ForwardDiff gives -2.4542495411512917

ForwardDiff gives -2.4542495411512917

Note above that the ForwardDiff.derivative() gives a value of the derivative up to machine precision

Now try using `Zygote.jl` Automatic Differentiation Package

```
In [9]: Zygote.gradient(f, x)
println("The true answer is ",d_true," Zygote gives ",Zygote.gradient(f,
```

The true answer is -2.4542495411512917 Zygote gives -2.4542495411512917
Zygote gives -2.4542495411512917

All the derivatives given by ForwardDiff.jl and Zygote.jl are accurate down to machine precision.

Define derivative functions for all two packages

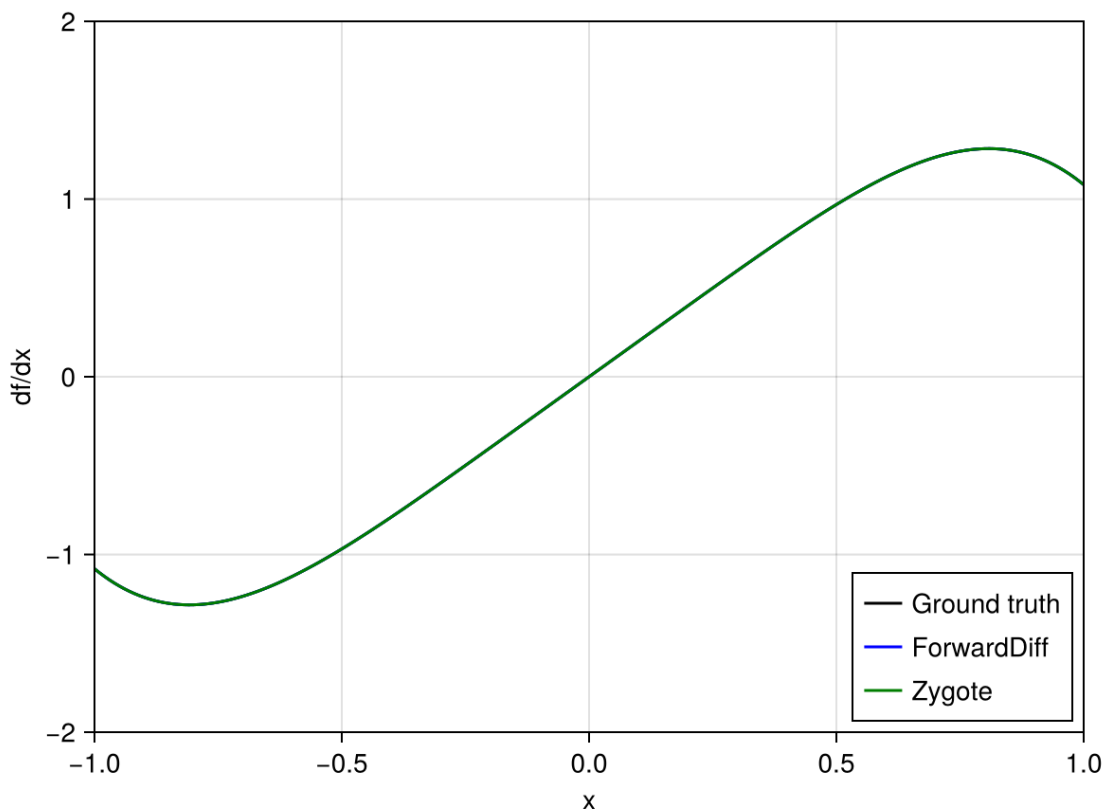
```
In [10]: dfForwardDiff(x)=ForwardDiff.derivative(f, x)
dfZygote(x)=Zygote.gradient(f, x)[1]
```

dfZygote (generic function with 1 method)

We can evaluate and plot df/dx for $-1 < x < 1$

```
In [11]: xplot=-1:0.01:1
fig = Figure()
ax = Axis(fig[1, 1], xlabel = "x", ylabel = "df/dx", limits=(-1.0,1.0,-2.0
lines!(xplot,2*xplot.*cos.(xplot.^2);color=:black,label="Ground truth")
lines!(xplot,dfForwardDiff.(xplot);color=:blue,label="ForwardDiff")
lines!(xplot,dfZygote.(xplot);color=:green,label="Zygote")

axislegend(ax,position = :rb)
fig
```



The plot above shows that ForwardDiff, Zygote gives exact values (to machine accuracy) of the derivatives for the whole range of x values.

In the first week of this course, we used the Newton-Raphson method to find the zero of the function

$$f(x) = 4\pi x - \frac{2}{x^2}$$

We used the `NonlinearSolve` in Julia to find x where $f(x) = 0$. Here we will use the Newton-Raphson method but I am going to use `ForwardDiff` to calculate the derivative (so that I do not need to know how to differentiate $f(x)$!). Recall that for this function, the value of x where $f(x) = 0$ is approximately $x = 0.541\dots$. We will use the Newton-Raphson method

$$x_{i+1} = x_i - f(x_i)/f'(x_i)$$

```
In [13]: f2(x)=4π*x-2/x^2
          x=1.0
          for i=1:10
              x=f2(x)/ForwardDiff.derivative(f2,x)
              println(x)
          end
```

Multivariate Function without any parameters

$$f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$
$$\frac{\partial f}{\partial x_1}(x_1, x_2) = 4x_1(x_1^2 + x_2 - 11) + 2(x_1 + x_2^2 - 7)$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) = 2(x_1^2 + x_2 - 11) + 4x_2(x_1 + x_2^2 - 7)$$

$$\frac{\partial^2 f}{\partial x_1^2}(x_1, x_2) = 12x_1^2 + 4x_2 - 42$$

$$\frac{\partial^2 f}{\partial x_1 \partial x_2}(x_1, x_2) = 4x_1 + 4x_2$$

$$\frac{\partial^2 f}{\partial x_2 \partial x_1}(x_1, x_2) = 4x_1 + 4x_2$$

$$\frac{\partial^2 f}{\partial x_2^2}(x_1, x_2) = 4x_1 + 12x_2^2 - 26$$

$$h(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

file:///Users/asho/Library/CloudStorage/OneDrive-TheUniversityofMelbourne/JuliaPrograms/PekingUniGlobexCourse/HTMLAndPDFs/04p01DifferentiationI...

Use `ForwardDiff` to calculate the gradient of the himmelblau function at $(x_1, x_2) = (0, 0)$.

```
In [18]: x0 = [0.0, 0.0]
ForwardDiff.gradient(himmelblau,x0)
```

```
2-element Vector{Float64}:
-14.0
-22.0
```

Exercise Ex02: Try using the `Zygote.gradient()` to calculate the gradient of the himmelblau function at $(x_1, x_2) = (0, 0)$

Using Gradient Descent with `ForwardDiff` to calculate the minimum of the Himmelblau function. You can see below, I do not need to manually figure out what the gradients are. I just use `ForwardDiff.gradient()` to calculate the (exact) gradients, $\partial h / \partial x_1$ and $\partial h / \partial x_2$

```
In [19]: struct Point2D
           x1::Float64
           x2::Float64
           fvalue::Float64
       end

x0=[-2.0,-2.0]#initial guess of x0
alpha=0.01
data_gd=[Point2D(x0...,himmelblau(x0))]

x=x0
for i in 1 : 50
    x-=alpha*ForwardDiff.gradient(himmelblau,x)
    push!(data_gd,Point2D(x...,himmelblau(x)))
end

data_gd
```

```
51-element Vector{Point2D}:
Point2D(-2.0, -2.0, 106.0)
Point2D(-2.62, -2.22, 62.404761919999984)
Point2D(-3.19223488, -2.5095020800000003, 26.184856197768234)
Point2D(-3.5381609903413658, -2.8340630130552853, 8.011765332690304)
Point2D(-3.6742112090087837, -3.0918679875433708, 1.5927632586276927)
Point2D(-3.7389311349776633, -3.217870523191624, 0.20441067649087527)
Point2D(-3.7668805073219027, -3.2625626716330705, 0.020376679773417165)
Point2D(-3.7754546966932847, -3.2770942749729803, 0.0018344361969869342)
Point2D(-3.778211411814951, -3.281366700658341, 0.00015965025359141713)
Point2D(-3.778974634603722, -3.282660320088421, 1.3752814215095311e-5)
:
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
Point2D(-3.779310253377747, -3.283185991286169, 7.888609052210118e-31)
```

The method of gradient descent converges to $(x_1, x_2) \approx (-3.779..., -3.283...)$

Function with parameters

We will now illustrate how you can use `ForwardDiff` or `Zygote` packages to calculate derivatives of function with many parameters. The important thing to note is that `ForwardDiff` can only differentiate functions with one argument. So if you have a function with many input, you need to turn it into a function with just one input. The one input can be a vector. The `Zygote` package can differentiate with respect to many variables (inputs).

First, define a function with two parameters W and b

$$NN(x; W, b) = \sin(Wx + b)$$

```
In [24]: function NN(x,W,b)
          sin(W*x+b)
        end
```

NN (generic function with 1 method)

`ForwardDiff` will fail if you try to use it on $NN(x, W, b)$. Try finding the gradient of NN with respect to x , W and b at $x = 1$, $W = 2$ and $b=3$

```
In [45]: ForwardDiff.gradient(NN,1,2,3)
```

```
MethodError: no method matching gradient(::typeof(NN), ::Int64, ::Int64,
::Int64)
```

The function `gradient` exists, but no method is defined for this combination of argument types.

Closest candidates are:

```
gradient(::Any, ::Real)
  @ ForwardDiff ~/.julia/packages/ForwardDiff/4J2wz/src/gradient.jl:46
gradient(::F, !Matched::StaticArraysCore.StaticArray) where F
  @ ForwardDiffStaticArraysExt ~/.julia/packages/ForwardDiff/4J2wz/ext/ForwardDiffStaticArraysExt.jl:36
gradient(::F, !Matched::StaticArraysCore.StaticArray, !Matched::ForwardDiff.GradientConfig) where F
  @ ForwardDiffStaticArraysExt ~/.julia/packages/ForwardDiff/4J2wz/ext/ForwardDiffStaticArraysExt.jl:37
...
```

Stacktrace:

```
[1] top-level scope
  @ ~/.Library/CloudStorage/OneDrive-TheUniversityofMelbourne/JuliaPrograms/PekingUniGlobexCourse/Chapter04DifferentiationAndBoundaryValueProblems/jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_Y151sZmlsZQ==.jl:1
```

Calculating

$$\frac{\partial NN}{\partial x}(x = 1, W = 2, b = 3) = W \cos(Wx + b) = 2 \cos(2 + 3) = 0.5673243709264525$$

```
In [51]: ForwardDiff.derivative((x)->NN(x,2,3),1.0)
```

```
0.5673243709264525
```

Calculating

$$\frac{\partial NN}{\partial W}(x = 1, W = 2, b = 3) = x \cos(Wx + b) = 1 \cos(2 + 3) = 0.28366218546322625$$

```
In [48]: ForwardDiff.derivative((W)->NN(1,W,3),2.0)
```

```
0.28366218546322625
```

Calculating

$$\frac{\partial NN}{\partial b}(x = 1, W = 2, b = 3) = \cos(Wx + b) = 1 \cos(2 + 3) = 0.2836621854632262$$

```
In [52]: ForwardDiff.derivative((b)->NN(1,2,b),3.0)
```

```
0.28366218546322625
```

Or you can just use the `Zygote` package

```
In [53]: Zygote.gradient(NN,1,2,3)
```

```
(0.5673243709264525, 0.28366218546322625, 0.28366218546322625)
```

You can also use the packages to calculate 2nd derivatives. The code below uses the `Zygote` package to calculate the 1st and 2nd derivatives of NN with respect to x . In the code below, we have used $W = 1$ and $b = 1$. $NNd1$ is the first derivative and $NNd2$ is the 2nd derivative.

```
In [56]: W=1
          b=0
          NNd1=(x)->Zygote.gradient(NN,x,W,b)[1]
          NNd2=(x)->Zygote.gradient(NNd1,x)[1]
```

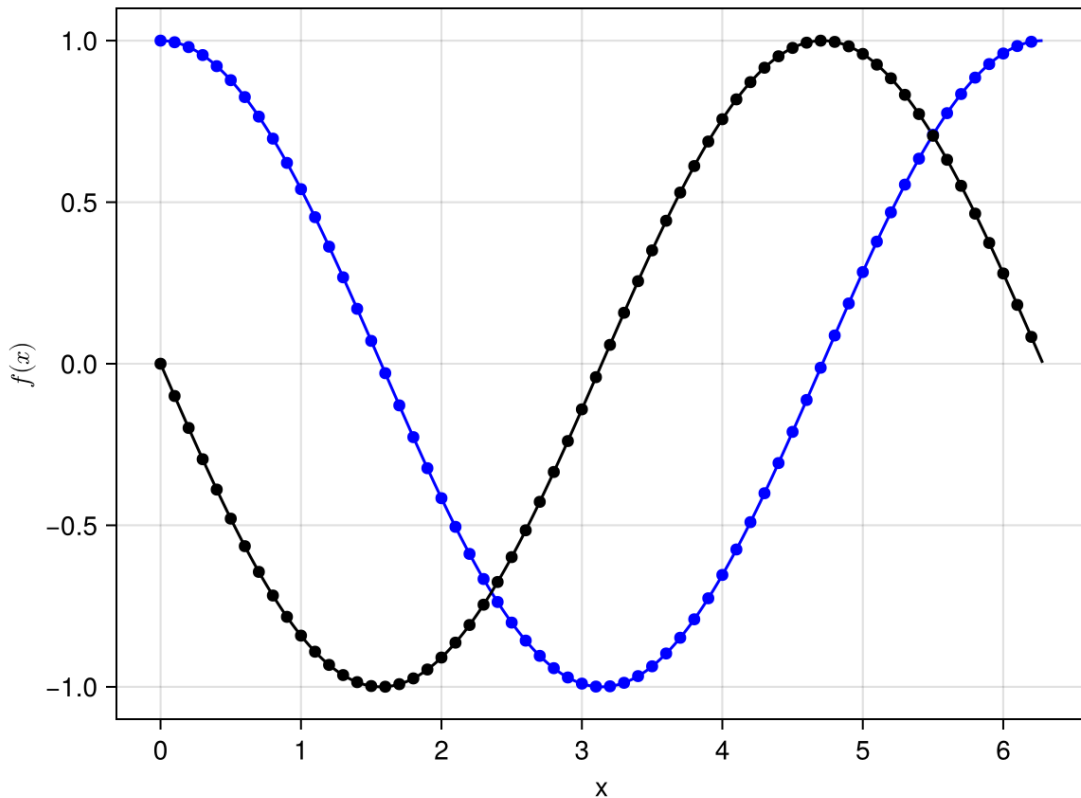
```
#76 (generic function with 1 method)
```

```
In [57]: xplot=Array(0:0.01:2π)

          xdata=Array(0:0.1:2π)
          W=1.0
          b=0.0
          fig = Figure()
          ax = Axis(fig[1, 1], xlabel = "x", ylabel = L"f(x)")
          lines!(ax,xplot,NNd1.(xplot);color=:blue)
          lines!(ax,xplot,NNd2.(xplot);color=:black)

          scatter!(ax,xdata,W*cos.(W*xdata.+b);color=:blue)
          scatter!(ax,xdata,-W*W*sin.(W*xdata.+b);color=:black)

          fig
```



In Notebook 3.5, we used the `Optimisation()` package in Julia to find the parameters of the model function

$$f_{model}(x) = a_1(1 - e^{a_2x})$$

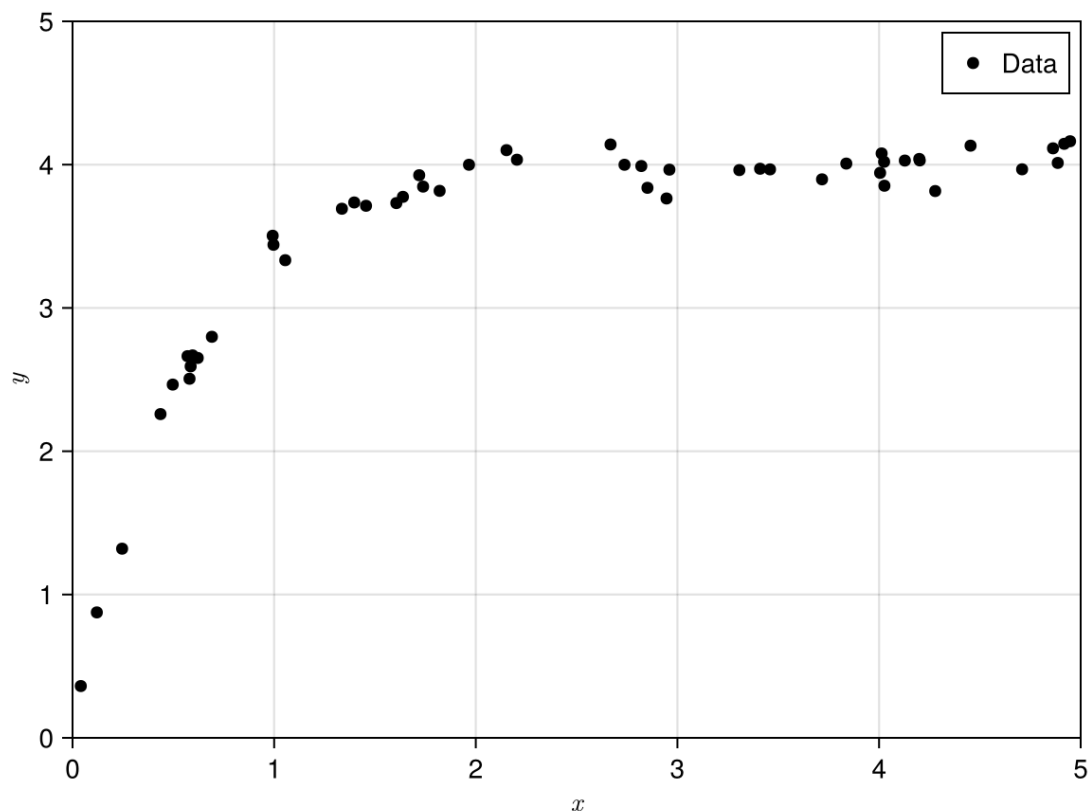
to fit a fictitious dataset. In this section of the notebook, we will get a better insight on what the `Optimisation()` package is doing.

Let's generate the fictitious dataset (again!) and plot the data to see how it looks like.

```
In [58]: N_SAMPLES=50 #50 data points
rng=Xoshiro(1) #my random number generator

x_samples=rand(rng,Uniform(0,5),N_SAMPLES) #generate random values of x b
y_noise=rand(rng,Normal(0.0,0.1),N_SAMPLES) #generate some noise, normal
y_samples=4*(1.0.-exp.(-1.8*x_samples))+y_noise; #y=2x+3+epsilon

fig = Figure()
ax = Axis(fig[1, 1], xlabel = L"x", ylabel = L"y", limits=(0,5,0,5))
scatter!(ax,x_samples,y_samples;label="Data",color=:black)
axislegend(ax;position = :rt)
fig
```



Define the model

$$f_{\text{model}}(\vec{a}, x) = a_1(1 - e^{a_2 x})$$

We will also define a custom loss function

$$S = \frac{1}{2} \sum_{i=1}^N (y_i - f_{\text{model}}(\vec{a}, x_i))^2$$

The idea is to find a_1 and a_2 such that we make S as small as possible. We will use standard gradient descent. To use this method, we need to find gradient $\frac{\partial S}{\partial a_1}$ and $\frac{\partial S}{\partial a_2}$ and we will use automatic differentiation to calculate the gradient.

We will define the model and the loss function.

```
In [59]: fmodel(a,x)=a[1]*(1.0.-exp.(a[2]*x)); # for this model a[1]=a and a[2]=b.
function custom_loss(parameters,x_samples)
    ŷ = fmodel(parameters, x_samples)
    0.5*sum((y_samples .- ŷ).^2)
end
a=[1.0,-1.0] #initial assumptions of the values of a0, a1
```

2-element Vector{Float64}:

```
1.0
-1.0
```

We will define a function S that is just a function of $\vec{a}$. Then we use `Zygote()` to calculate the gradient with respect to $\vec{a}$, $\partial S / \partial a_1$ and $\partial S / \partial a_2$

```
In [60]: S=(a)->custom_loss(a,x_samples)
```

```
Zygote.gradient(S, a)[1]
```

2-element Vector{Float64}:

```
-113.90018346559744  
 25.972734381952076
```

Use gradient descent to minimize $S(\vec{a})$

```
In [61]: α=0.01 #learning rate  
for i=1:500  
    a-=α * Zygote.gradient(S,a)[1]  
    println(a)  
end
```

[2.1390018346559745, -1.2597273438195207]
[2.8987269538320937, -1.4969371995729637]
[3.366404392779467, -1.6375194285298376]
[3.642257471617602, -1.7181989633830088]
[3.801882722998297, -1.7641794083694813]
[3.8933764991482684, -1.7903362982424835]
[3.945553758140763, -1.8052169571373649]
[3.9752248457675963, -1.813693557831972]
[3.992068343102892, -1.8185351132858958]
[4.001618185207118, -1.8213133617986053]
[4.007026598148098, -1.8229199449951736]
[4.01008539004146, -1.8238605712596616]
[4.011811873303483, -1.8244220518462573]
[4.012783245965075, -1.8247670723103537]
[4.013326865565147, -1.8249879405206282]
[4.013628345180858, -1.825137062221884]
[4.013792907451754, -1.825244218956136]
[4.013880190430729, -1.8253263631367602]
[4.013923987015601, -1.8253931709079485]
[4.013943437907328, -1.8254501901135394]
[4.0139493778059725, -1.8255006249787296]
[4.013947933376313, -1.8255463479755794]
[4.0139425630297305, -1.8255884735945358]
[4.013935213293178, -1.82562768376534]
[4.013926974340671, -1.8256644124866808]
[4.01391845158757, -1.8256989506367831]
[4.0139099763159525, -1.8257315055285475]
[4.013901725045158, -1.8257622348027618]
[4.013893787170025, -1.8257912657668411]
[4.013886203272912, -1.825818706476041]
[4.0138789868119105, -1.8258446521271268]
[4.013872136386399, -1.825869188788615]
[4.013865642662346, -1.825892395615359]
[4.013859492271652, -1.8259143461984626]
[4.013853669997505, -1.825935109419852]
[4.013848159989399, -1.8259547500211648]
[4.013842946429387, -1.8259733290060787]
[4.013838013888445, -1.8259909039438627]
[4.013833347508344, -1.826007529212818]
[4.013828933085686, -1.826023256205753]
[4.013824757101535, -1.8260381335102627]
[4.01382080672118, -1.8260522070712495]
[4.013817069777902, -1.8260655203400973]
[4.013813534748559, -1.826078114413175]
[4.013810190725345, -1.826090028161359]
[4.013807027386191, -1.8261012983516922]
[4.013804034965117, -1.826111959761965]
[4.013801204223273, -1.826122045288806]
[4.013798526421022, -1.826131586049752]
[4.0137959932912475, -1.82614061147969]
[4.0137935970139464, -1.826149149422017]
[4.013791330192103, -1.826157226214822]
[4.0137891858288315, -1.8261648667723807]
[4.01378715730572, -1.8261720946622169]
[4.013785238362348, -1.826178932177983]
[4.013783423076895, -1.8261854004083853]
[4.0137817058478085, -1.8261915193023697]
[4.0137800813764555, -1.8261973077307774]
[4.013778544650725, -1.8262027835446561]
[4.01377709092951, -1.8262079636304145]

[4.01377571572804, -1.8262128639619868]
[4.0137744148040015, -1.8262174996501745]
[4.013773184144415, -1.8262218849893153]
[4.013772019953228, -1.8262260335014249]
[4.013770918639573, -1.8262299579779506]
[4.013769876806669, -1.8262336705192643]
[4.013768891241316, -1.8262371825720183]
[4.013767958903972, -1.8262405049644799]
[4.013767076919353, -1.8262436479399544]
[4.013766242567554, -1.8262466211883999]
[4.013765453275643, -1.8262494338763315]
[4.013764706609717, -1.8262520946751093]
[4.01376400026738, -1.8262546117876945]
[4.0137633320706385, -1.826256992973961]
[4.0137626999591705, -1.8262592455746358]
[4.013762101983969, -1.826261376533948]
[4.0137615363013195, -1.8262633924210496]
[4.013761001167115, -1.826265299450282]
[4.013760494931467, -1.826267103500343]
[4.013760016033615, -1.8262688101324225]
[4.0137595629971115, -1.826270424607353]
[4.013759134425263, -1.8262719519018362]
[4.013758728996822, -1.8262733967237927]
[4.013758345461908, -1.8262747635268801]
[4.013757982638154, -1.8262760565242282]
[4.013757639407057, -1.8262772797014317]
[4.013757314710527, -1.8262784368288405]
[4.013757007547624, -1.8262795314731857]
[4.01375671697147, -1.8262805670085784]
[4.013756442086327, -1.8262815466269136]
[4.0137561820448395, -1.826282473347711]
[4.013755936045411, -1.8262833500274263]
[4.013755703329741, -1.8262841793682578]
[4.013755483180484, -1.8262849639264782]
[4.013755274919033, -1.826285706120315]
[4.013755077903434, -1.8262864082374066]
[4.013754891526398, -1.8262870724418554]
[4.013754715213436, -1.8262877007808986]
[4.013754548421082, -1.826288295191221]
[4.013754390635223, -1.826288857504924]
[4.013754241369507, -1.8262893894551748]
[4.013754100163847, -1.8262898926815485]
[4.013753966583002, -1.8262903687350833]
[4.013753840215239, -1.826290819083061]
[4.013753720671054, -1.8262912451135305]
[4.01375360758198, -1.8262916481395868]
[4.013753500599448, -1.8262920294034184]
[4.013753399393711, -1.8262923900801367]
[4.01375330365283, -1.826292731281398]
[4.013753213081712, -1.82629305405883]
[4.013753127401195, -1.8262933594072746]
[4.013753046347197, -1.826293648267853]
[4.013752969669892, -1.8262939215308671]
[4.013752897132948, -1.8262941800385444]
[4.013752828512793, -1.8262944245876331]
[4.013752763597929, -1.8262946559318587]
[4.013752702188278, -1.8262948747842465]
[4.013752644094567, -1.8262950818193202]
[4.013752589137742, -1.8262952776751809]
[4.013752537148418, -1.8262954629554728]

[4.013752487966359, -1.8262956382312456]
[4.013752441439978, -1.826295804042713]
[4.0137523974258755, -1.8262959609009188]
[4.013752355788396, -1.8262961092893115]
[4.0137523163992075, -1.8262962496652346]
[4.0137522791369085, -1.8262963824613365]
[4.013752243886653, -1.8262965080869034]
[4.013752210539797, -1.8262966269291212]
[4.0137521789935615, -1.8262967393542688]
[4.013752149150719, -1.8262968457088475]
[4.01375212091929, -1.826296946320648]
[4.013752094212264, -1.8262970414997615]
[4.013752068947328, -1.826297131539535]
[4.013752045046613, -1.826297216717476]
[4.013752022436456, -1.826297297296107]
[4.01375200104717, -1.826297373523775]
[4.013751980812832, -1.826297445635418]
[4.01375196167108, -1.8262975138532869]
[4.013751943562916, -1.8262975783876323]
[4.01375192643253, -1.8262976394373518]
[4.013751910227126, -1.8262976971906033]
[4.013751894896757, -1.826297751825385]
[4.0137518803941745, -1.826297803510083]
[4.013751866674681, -1.826297852403992]
[4.013751853695992, -1.8262978986578042]
[4.013751841418107, -1.8262979424140764]
[4.013751829803184, -1.8262979838076665]
[4.013751818815426, -1.8262980229661514]
[4.0137518084209685, -1.8262980600102188]
[4.013751798587775, -1.8262980950540395]
[4.013751789285538, -1.82629812820562]
[4.013751780485589, -1.8262981595671348]
[4.0137517721608065, -1.8262981892352408]
[4.013751764285532, -1.8262982173013762]
[4.013751756835494, -1.8262982438520416]
[4.0137517497877315, -1.8262982689690672]
[4.013751743120523, -1.826298292729864]
[4.013751736813321, -1.826298315207664]
[4.013751730846684, -1.826298336471744]
[4.013751725202224, -1.826298356587641]
[4.013751719862545, -1.8262983756173523]
[4.013751714811189, -1.8262983936195283]
[4.013751710032588, -1.8262984106496523]
[4.013751705512014, -1.8262984267602114]
[4.0137517012355355, -1.8262984420008588]
[4.013751697189971, -1.8262984564185667]
[4.013751693362852, -1.8262984700577707]
[4.013751689742383, -1.8262984829605073]
[4.013751686317407, -1.8262984951665433]
[4.013751683077368, -1.8262985067134976]
[4.0137516800122786, -1.8262985176369584]
[4.013751677112693, -1.8262985279705921]
[4.013751674369675, -1.8262985377462473]
[4.013751671774769, -1.8262985469940527]
[4.013751669319979, -1.8262985557425104]
[4.013751666997739, -1.8262985640185834]
[4.013751664800892, -1.8262985718477784]
[4.013751662722666, -1.8262985792542255]
[4.013751660756657, -1.8262985862607515]
[4.013751658896805, -1.8262985928889506]

[4.013751657137378, -1.8262985991592513]
[4.013751655472954, -1.8262986050909786]
[4.013751653898403, -1.8262986107024142]
[4.013751652408872, -1.8262986160108527]
[4.01375165099977, -1.826298621032655]
[4.013751649666754, -1.8262986257832985]
[4.013751648405717, -1.8262986302774245]
[4.013751647212771, -1.826298634528884]
[4.013751646084239, -1.8262986385507805]
[4.013751645016644, -1.826298642355509]
[4.013751644006695, -1.8262986459547963]
[4.01375164305128, -1.826298649359735]
[4.0137516421474535, -1.8262986525808196]
[4.013751641292431, -1.8262986556279774]
[4.0137516404835765, -1.8262986585105998]
[4.0137516397183965, -1.8262986612375711]
[4.013751638994534, -1.826298663817296]
[4.013751638309757, -1.8262986662577252]
[4.013751637661956, -1.8262986685663802]
[4.0137516370491335, -1.8262986707503763]
[4.013751636469402, -1.8262986728164445]
[4.013751635920973, -1.8262986747709526]
[4.013751635402157, -1.8262986766199245]
[4.013751634911356, -1.8262986783690587]
[4.013751634447056, -1.826298680023746]
[4.013751634007826, -1.826298681589086]
[4.013751633592313, -1.8262986830699035]
[4.013751633199236, -1.8262986844707623]
[4.013751632827384, -1.82629868579598]
[4.0137516324756115, -1.8262986870496407]
[4.013751632142833, -1.8262986882356083]
[4.013751631828023, -1.826298689357538]
[4.013751631530211, -1.8262986904188876]
[4.013751631248481, -1.8262986914229282]
[4.013751630981963, -1.8262986923727542]
[4.0137516307298355, -1.8262986932712932]
[4.013751630491322, -1.8262986941213144]
[4.013751630265688, -1.8262986949254376]
[4.013751630052237, -1.8262986956861411]
[4.0137516298503115, -1.8262986964057693]
[4.013751629659289, -1.8262986970865402]
[4.013751629478581, -1.826298697730552]
[4.013751629307632, -1.8262986983397895]
[4.013751629145912, -1.8262986989161305]
[4.013751628992925, -1.826298699461351]
[4.0137516288481985, -1.8262986999771318]
[4.013751628711287, -1.8262987004650622]
[4.013751628581768, -1.826298700926646]
[4.013751628459243, -1.826298701363306]
[4.013751628343333, -1.826298701776388]
[4.013751628233682, -1.8262987021671653]
[4.013751628129952, -1.826298702536842]
[4.013751628031823, -1.8262987028865574]
[4.013751627938992, -1.8262987032173894]
[4.013751627851175, -1.8262987035303577]
[4.013751627768099, -1.826298703826427]
[4.013751627689508, -1.8262987041065095]
[4.013751627615162, -1.8262987043714687]
[4.0137516275448295, -1.826298704622121]
[4.013751627478295, -1.826298704859239]

[4.0137516274153535, -1.8262987050835535]
[4.01375162735581, -1.8262987052957558]
[4.013751627299482, -1.8262987054964999]
[4.0137516272461955, -1.8262987056864046]
[4.013751627195786, -1.8262987058660551]
[4.013751627148098, -1.8262987060360052]
[4.013751627102986, -1.8262987061967786]
[4.013751627060309, -1.826298706348871]
[4.0137516270199365, -1.8262987064927507]
[4.013751626981744, -1.8262987066288616]
[4.013751626945614, -1.826298706757623]
[4.013751626911435, -1.8262987068794319]
[4.013751626879102, -1.8262987069946635]
[4.013751626848514, -1.826298707103673]
[4.013751626819578, -1.8262987072067962]
[4.013751626792204, -1.8262987073043513]
[4.013751626766308, -1.8262987073966388]
[4.013751626741811, -1.826298707483943]
[4.013751626718637, -1.8262987075665331]
[4.013751626696714, -1.8262987076446635]
[4.013751626675974, -1.8262987077185753]
[4.013751626656354, -1.826298707788496]
[4.013751626637794, -1.8262987078546413]
[4.013751626620237, -1.826298707917215]
[4.013751626603627, -1.82629870797641]
[4.013751626587914, -1.8262987080324087]
[4.0137516265730495, -1.8262987080853836]
[4.013751626558987, -1.826298708135498]
[4.013751626545685, -1.8262987081829065]
[4.0137516265331, -1.826298708227755]
[4.013751626521195, -1.826298708270182]
[4.013751626509933, -1.8262987083103182]
[4.013751626499279, -1.8262987083482871]
[4.0137516264892, -1.8262987083842057]
[4.0137516264796655, -1.826298708418185]
[4.013751626470646, -1.8262987084503295]
[4.013751626462113, -1.8262987084807383]
[4.013751626454042, -1.826298708509505]
[4.013751626446406, -1.8262987085367186]
[4.013751626439182, -1.8262987085624627]
[4.013751626432349, -1.8262987085868165]
[4.013751626425884, -1.8262987086098554]
[4.013751626419769, -1.8262987086316504]
[4.013751626413983, -1.8262987086522686]
[4.01375162640851, -1.8262987086717735]
[4.013751626403333, -1.8262987086902251]
[4.0137516263984345, -1.8262987087076803]
[4.013751626393801, -1.826298708724193]
[4.013751626389418, -1.8262987087398141]
[4.013751626385271, -1.8262987087545919]
[4.013751626381349, -1.8262987087685716]
[4.013751626377638, -1.8262987087817963]
[4.013751626374128, -1.826298708794307]
[4.013751626370807, -1.8262987088061422]
[4.013751626367665, -1.8262987088173384]
[4.0137516263646935, -1.8262987088279299]
[4.0137516263618815, -1.8262987088379496]
[4.0137516263592214, -1.8262987088474283]
[4.013751626356705, -1.8262987088563951]
[4.013751626354325, -1.8262987088648779]

[4.013751626352073, -1.8262987088729026]
[4.0137516263499435, -1.826298708880494]
[4.013751626347928, -1.8262987088876754]
[4.013751626346022, -1.826298708894469]
[4.013751626344219, -1.826298708900896]
[4.013751626342513, -1.8262987089069758]
[4.013751626340899, -1.8262987089127274]
[4.013751626339372, -1.8262987089181684]
[4.013751626337928, -1.8262987089233156]
[4.013751626336562, -1.8262987089281848]
[4.01375162633527, -1.8262987089327911]
[4.013751626334047, -1.8262987089371487]
[4.01375162633289, -1.826298708941271]
[4.013751626331795, -1.8262987089451708]
[4.013751626330761, -1.8262987089488598]
[4.013751626329781, -1.8262987089523497]
[4.013751626328855, -1.8262987089556513]
[4.013751626327978, -1.8262987089587746]
[4.013751626327149, -1.826298708961729]
[4.013751626326365, -1.8262987089645242]
[4.013751626325623, -1.8262987089671683]
[4.013751626324922, -1.8262987089696696]
[4.013751626324257, -1.826298708972036]
[4.0137516263236295, -1.8262987089742744]
[4.013751626323035, -1.826298708976392]
[4.013751626322473, -1.8262987089783953]
[4.013751626321941, -1.8262987089802905]
[4.013751626321438, -1.8262987089820832]
[4.013751626320962, -1.8262987089837792]
[4.013751626320512, -1.8262987089853837]
[4.013751626320086, -1.8262987089869014]
[4.013751626319683, -1.8262987089883373]
[4.013751626319302, -1.8262987089896956]
[4.013751626318942, -1.8262987089909806]
[4.013751626318601, -1.826298708992196]
[4.013751626318278, -1.826298708993346]
[4.013751626317973, -1.8262987089944338]
[4.013751626317684, -1.826298708995463]
[4.0137516263174104, -1.8262987089964364]
[4.013751626317152, -1.8262987089973575]
[4.013751626316908, -1.8262987089982288]
[4.013751626316677, -1.826298708999053]
[4.013751626316457, -1.8262987089998326]
[4.0137516263162505, -1.8262987090005702]
[4.013751626316055, -1.8262987090012681]
[4.0137516263158695, -1.8262987090019283]
[4.0137516263156945, -1.8262987090025526]
[4.013751626315528, -1.8262987090031433]
[4.013751626315372, -1.8262987090037022]
[4.013751626315224, -1.8262987090042309]
[4.013751626315083, -1.826298709004731]
[4.01375162631495, -1.826298709005204]
[4.013751626314825, -1.8262987090056517]
[4.013751626314706, -1.8262987090060752]
[4.013751626314593, -1.8262987090064757]
[4.013751626314487, -1.8262987090068548]
[4.013751626314386, -1.8262987090072131]
[4.013751626314291, -1.8262987090075522]
[4.0137516263142015, -1.826298709007873]
[4.013751626314116, -1.8262987090081766]

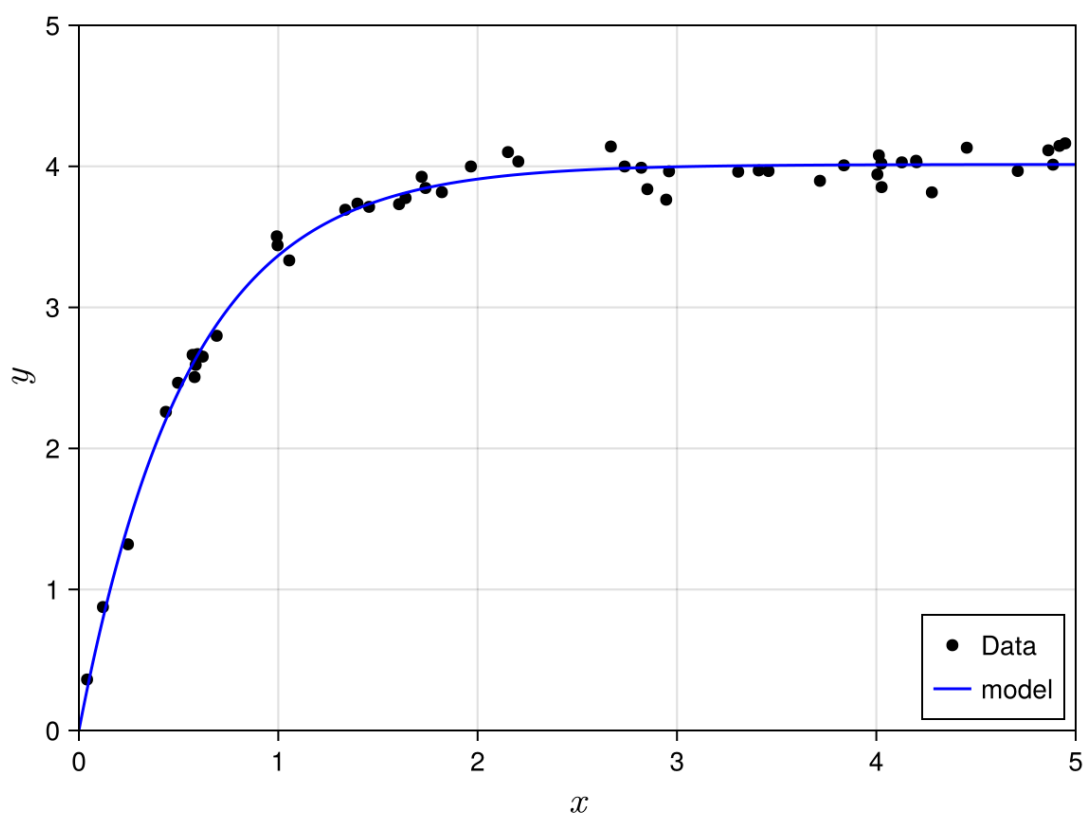
[4.013751626314035, -1.8262987090084637]
[4.013751626313959, -1.8262987090087353]
[4.013751626313887, -1.8262987090089922]
[4.013751626313819, -1.8262987090092353]
[4.013751626313755, -1.8262987090094653]
[4.013751626313693, -1.8262987090096827]
[4.013751626313636, -1.8262987090098886]
[4.0137516263135815, -1.8262987090100833]
[4.01375162631353, -1.8262987090102674]
[4.013751626313481, -1.8262987090104414]
[4.013751626313435, -1.8262987090106062]
[4.013751626313391, -1.826298709010762]
[4.01375162631335, -1.8262987090109095]
[4.013751626313311, -1.826298709011049]
[4.013751626313273, -1.8262987090111809]
[4.013751626313239, -1.8262987090113056]
[4.013751626313206, -1.8262987090114238]
[4.013751626313174, -1.8262987090115355]
[4.0137516263131445, -1.8262987090116412]
[4.013751626313116, -1.826298709011741]
[4.0137516263130895, -1.8262987090118357]
[4.013751626313065, -1.8262987090119251]
[4.013751626313041, -1.8262987090120097]
[4.013751626313018, -1.82629870901209]
[4.013751626312997, -1.8262987090121656]
[4.013751626312977, -1.8262987090122373]
[4.013751626312958, -1.826298709012305]
[4.01375162631294, -1.8262987090123692]
[4.013751626312923, -1.8262987090124299]
[4.013751626312907, -1.8262987090124871]
[4.013751626312892, -1.8262987090125413]
[4.013751626312877, -1.8262987090125926]
[4.013751626312864, -1.8262987090126412]
[4.0137516263128505, -1.8262987090126872]
[4.013751626312838, -1.8262987090127307]
[4.013751626312827, -1.8262987090127718]
[4.013751626312816, -1.8262987090128107]
[4.013751626312805, -1.8262987090128475]
[4.0137516263127955, -1.8262987090128824]
[4.013751626312787, -1.8262987090129155]
[4.013751626312778, -1.8262987090129466]
[4.01375162631277, -1.826298709012976]
[4.013751626312762, -1.826298709013004]
[4.013751626312755, -1.8262987090130305]
[4.0137516263127475, -1.8262987090130554]
[4.01375162631274, -1.826298709013079]
[4.013751626312734, -1.8262987090131013]
[4.013751626312728, -1.8262987090131224]
[4.013751626312723, -1.8262987090131424]
[4.013751626312717, -1.8262987090131613]
[4.013751626312712, -1.8262987090131793]
[4.0137516263127075, -1.8262987090131964]
[4.013751626312703, -1.8262987090132123]
[4.013751626312699, -1.8262987090132274]
[4.013751626312695, -1.8262987090132419]
[4.013751626312692, -1.8262987090132554]
[4.013751626312688, -1.8262987090132683]
[4.0137516263126845, -1.8262987090132803]
[4.013751626312681, -1.8262987090132918]
[4.013751626312678, -1.8262987090133027]

[4.013751626312676, -1.826298709013313]
[4.013751626312673, -1.8262987090133227]
[4.01375162631267, -1.8262987090133318]
[4.013751626312668, -1.8262987090133405]
[4.013751626312665, -1.8262987090133487]
[4.013751626312663, -1.8262987090133564]
[4.013751626312661, -1.8262987090133638]
[4.01375162631266, -1.8262987090133707]
[4.013751626312658, -1.826298709013377]
[4.013751626312656, -1.8262987090133833]
[4.013751626312654, -1.826298709013389]
[4.0137516263126525, -1.8262987090133946]
[4.013751626312651, -1.8262987090134]
[4.013751626312649, -1.826298709013405]
[4.013751626312648, -1.8262987090134097]
[4.013751626312647, -1.8262987090134142]
[4.013751626312645, -1.8262987090134184]
[4.0137516263126445, -1.8262987090134224]
[4.013751626312644, -1.8262987090134262]
[4.013751626312643, -1.8262987090134297]
[4.013751626312642, -1.826298709013433]
[4.013751626312641, -1.8262987090134362]
[4.01375162631264, -1.8262987090134393]
[4.013751626312639, -1.8262987090134422]
[4.013751626312638, -1.8262987090134448]
[4.013751626312637, -1.8262987090134475]
[4.0137516263126365, -1.82629870901345]
[4.013751626312636, -1.8262987090134521]
[4.013751626312635, -1.8262987090134544]
[4.013751626312635, -1.8262987090134566]
[4.013751626312634, -1.8262987090134586]
[4.013751626312634, -1.8262987090134604]
[4.013751626312633, -1.8262987090134621]
[4.013751626312633, -1.826298709013464]
[4.013751626312632, -1.8262987090134655]
[4.013751626312632, -1.826298709013467]
[4.013751626312631, -1.8262987090134684]
[4.013751626312631, -1.8262987090134697]
[4.013751626312631, -1.826298709013471]
[4.01375162631263, -1.8262987090134721]
[4.01375162631263, -1.8262987090134732]
[4.01375162631263, -1.8262987090134744]
[4.013751626312629, -1.8262987090134752]
[4.013751626312629, -1.8262987090134761]
[4.013751626312629, -1.826298709013477]
[4.013751626312629, -1.826298709013478]
[4.0137516263126285, -1.8262987090134786]
[4.0137516263126285, -1.8262987090134792]
[4.0137516263126285, -1.82629870901348]
[4.0137516263126285, -1.8262987090134806]
[4.0137516263126285, -1.8262987090134812]
[4.013751626312628, -1.8262987090134817]
[4.013751626312628, -1.8262987090134823]
[4.013751626312628, -1.8262987090134828]
[4.013751626312628, -1.8262987090134832]
[4.013751626312628, -1.8262987090134837]
[4.013751626312628, -1.8262987090134841]
[4.013751626312628, -1.8262987090134846]
[4.013751626312628, -1.826298709013485]
[4.013751626312627, -1.8262987090134855]

```
[4.013751626312627, -1.826298709013486]
[4.013751626312627, -1.8262987090134863]
[4.013751626312627, -1.8262987090134866]
[4.013751626312627, -1.8262987090134868]
[4.013751626312627, -1.826298709013487]
[4.013751626312627, -1.8262987090134872]
[4.013751626312627, -1.8262987090134875]
[4.013751626312627, -1.8262987090134877]
[4.013751626312627, -1.826298709013488]
[4.013751626312627, -1.8262987090134881]
[4.013751626312627, -1.8262987090134883]
[4.013751626312627, -1.8262987090134886]
[4.013751626312627, -1.8262987090134888]
[4.013751626312626, -1.826298709013489]
[4.013751626312626, -1.8262987090134892]
[4.013751626312626, -1.8262987090134895]
[4.013751626312626, -1.8262987090134897]
[4.013751626312626, -1.82629870901349]
[4.013751626312626, -1.8262987090134901]
[4.013751626312626, -1.8262987090134901]
```

Once we have found the optimum values of a_1 and a_2 , we can plot the model.

```
In [62]: fig = Figure()
ax = Axis(fig[1, 1], xlabel = L"x", ylabel = L"y", limits=(0,5,0,5),
          xlabelsize = 20, # Set X label size
          ylabelsize = 20)
scatter!(ax,x_samples,y_samples;label="Data",color=:black)
lines!(ax,xplot,fmodel(a,xplot);label="model",color=:blue)
axislegend(ax;position = :rb)
fig
```



Good agreement!

Vector (vector-valued) functions of vector

Vector functions are a collection of scalar valued functions of many variables. For many applications, you have a vector functions that you need to differentiate. I will illustrate the derivative of vector functions with the example below

$$\vec{f}(\vec{x})$$

$$f_i(\vec{x}) = \begin{cases} x_i - 1 & i = 1 \\ x_i - x_{i-1}^2 & i = 2..N - 1 \\ x_i - x_{i-1} & i = N \end{cases}$$

For $N = 5$, and if we assume that $x_1 = x_2 = x_3 = x_4 = x_5 = 2$ this function looks like

$$\begin{pmatrix} f_1(x_1, x_2, x_3, x_4, x_5) \\ f_2(x_1, x_2, x_3, x_4, x_5) \\ f_3(x_1, x_2, x_3, x_4, x_5) \\ f_4(x_1, x_2, x_3, x_4, x_5) \\ f_5(x_1, x_2, x_3, x_4, x_5) \end{pmatrix} = \begin{pmatrix} x_1 - 1 \\ x_2 - x_1^2 \\ x_3 - x_2^2 \\ x_4 - x_3^2 \\ x_5 - x_4 \end{pmatrix} = \begin{pmatrix} 1 \\ -2 \\ -2 \\ -2 \\ 0 \end{pmatrix}$$

The Jacobian of vector functions, is defined as the derivative of each component of the vector with respect to each variable. For example, the Jacobian $\partial \vec{f} / \partial \vec{x}$ for $N = 5$ is a 5×5 matrix. If $x_1 = x_2 = x_3 = x_4 = x_5 = 2$

$$\begin{aligned} \partial \vec{f} / \partial \vec{x} &= \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} & \frac{\partial f_1}{\partial x_4} & \frac{\partial f_1}{\partial x_5} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} & \frac{\partial f_2}{\partial x_4} & \frac{\partial f_2}{\partial x_5} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} & \frac{\partial f_3}{\partial x_4} & \frac{\partial f_3}{\partial x_5} \\ \frac{\partial f_4}{\partial x_1} & \frac{\partial f_4}{\partial x_2} & \frac{\partial f_4}{\partial x_3} & \frac{\partial f_4}{\partial x_4} & \frac{\partial f_4}{\partial x_5} \\ \frac{\partial f_5}{\partial x_1} & \frac{\partial f_5}{\partial x_2} & \frac{\partial f_5}{\partial x_3} & \frac{\partial f_5}{\partial x_4} & \frac{\partial f_5}{\partial x_5} \end{bmatrix} = \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -2x_1 & 1 & 0 & 0 & 0 \\ 0 & -2x_2 & 1 & 0 & 0 \\ 0 & 0 & -2x_3 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix} = \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -4 & 1 & 0 & 0 & 0 \\ 0 & -4 & 1 & 0 & 0 \\ 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix} \end{aligned}$$

\$\$

I will now show how you can use `Zygote` or `ForwardDiff` to find the Jacobian of a $\vec{f}(\vec{x})$. First we need to define $\vec{f}$. In the example below, I will define $\vec{f}$ in two different ways, in the function `fvec1()` and `fvec2()`. You will see that only one of them can be differentiated using `Zygote` and both of them works with `ForwardDiff`.

```
In [63]: function fvec1(x)
           df=similar(x)
           df[1]=x[1]-1
           for i=2:length(x)-1
               df[i]=x[i]-x[i-1]^2
           end
           df[end]=x[end]-x[end-1]
           return df
       end
```

fvec1 (generic function with 1 method)

```
In [64]: fvec2(x)=[x[1]-1;
                  x[2:end-1].-x[1:end-2].^2;
                  x[end]-x[end-1]]
```

fvec2 (generic function with 1 method)

```
In [68]: N=5
          x=2*ones(N)
          fvec1(x)
```

5-element Vector{Float64}:

```
 1.0
-2.0
-2.0
-2.0
 0.0
```

```
In [69]: fvec1(x).-fvec2(x)
```

5-element Vector{Float64}:

```
 0.0
 0.0
 0.0
 0.0
 0.0
```

`fvec1()` and `fvec2()` are exactly the same function. However, you will see in the code below that one of them can be differentiated using `Zygote` but both of them works with `ForwardDiff`.

Now we use `Zygote` to find the Jacobian of $\vec{f}(\vec{x})$

```
In [70]: Zygote.jacobian(fvec1,x)[1]
```

Mutating arrays is not supported -- called `setindex!(Vector{Float64}, ...)`

This error occurs when you ask Zygote to differentiate operations that change

the elements of arrays in place (e.g. setting values with `x .= ...`)

Possible fixes:

- avoid mutating operations (preferred)
- or read the documentation and solutions for this error

<https://fluxml.ai/Zygote.jl/latest/limitations>

Stacktrace:

```
[1] error(s::String)
      @ Base ./error.jl:35

[2] _throw_mutation_error(f::Function, args::Vector{Float64})
      @ Zygote ~/.julia/packages/Zygote/Pm9ID/src/lib/array.jl:70

[3] (::Zygote.var"#588#589"{Vector{Float64}})::Nothing
      @ Zygote ~/.julia/packages/Zygote/Pm9ID/src/lib/array.jl:82

[4] (::Zygote.var"#2699#back#590"{Zygote.var"#588#589"{Vector{Float64}}})::Nothing
      @ Zygote ~/.julia/packages/ZygoteRules/CkVIK/src/adjoint.jl:72

[5] fvec1
      @ ~/Library/CloudStorage/OneDrive-TheUniversityofMelbourne/JuliaPrograms/PekingUniGlobexCourse/Chapter04DifferentiationAndBoundaryValueProblems/jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_Y121sZmlsZQ==.jl:7 [inlined]

[6] (::Zygote.Pullback{Tuple{typeof(fvec1), Vector{Float64}}, Any})(Δ::Vector{Float64})
      @ Zygote ~/.julia/packages/Zygote/Pm9ID/src/compiler/interface2.jl:0

[7] #308
      @ ~/.julia/packages/Zygote/Pm9ID/src/lib/lib.jl:214 [inlined]

[8] (::Zygote.var"#2197#back#310"{Zygote.var"#308#309"{Tuple{Tuple{Nothing}}, Zygote.Pullback{Tuple{typeof(fvec1), Vector{Float64}}, Any}}})(Δ::Vector{Float64})
```

```

@ Zygote ~/.julia/packages/ZygoteRules/CkVIK/src/adjoint.jl:72

[9] call_composed

@ ./operators.jl:1054 [inlined]

[10] (::Zygote.Pullback{Tuple{typeof(Base.call_composed), Tuple{typeof(fvec1)}, Tuple{Vector{Float64}}, @Kwargs{}}}, Any})(Δ::Vector{Float64})

@ Zygote ~/.julia/packages/Zygote/Pm9ID/src/compiler/interface2.jl:0

[11] call_composed

@ ./operators.jl:1053 [inlined]

[12] #_#113

@ ./operators.jl:1050 [inlined]

[13] (::Zygote.Pullback{Tuple{Base.var"##_#113", @Kwargs{}}}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, Vector{Float64}}, Tuple{Zygote.Pullback{Tuple{typeof(Base.call_composed), Tuple{typeof(Zygote._jvec), typeof(fvec1)}, Tuple{Vector{Float64}}, @Kwargs{}}}, Tuple{Zygote.var"#2057#back#230"{Zygote.var"#back#228"{2, 1, Zygote.Context{false}, typeof(Zygote._jvec)}}, Zygote.Pullback{Tuple{typeof(Zygote._jvec), Vector{Float64}}, Tuple{Zygote.Pullback{Tuple{typeof(vec), Vector{Float64}}, Tuple{}}}}, Zygote.var"#2169#back#298"{Zygote.var"#294#297"}}, Zygote.Pullback{Tuple{typeof(Base.call_composed), Tuple{typeof(fvec1)}, Tuple{Vector{Float64}}, @Kwargs{}}}, Any}}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, Tuple{Zygote.var"#2208#back#320"{Zygote.var"#back#319"{:outer, Zygote.Context{false}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, typeof(Zygote._jvec)}}, Zygote.var"#2208#back#320"{Zygote.var"#back#319"{:inner, Zygote.Context{false}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, typeof(fvec1)}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), typeof(fvec1)}, Tuple{Zygote.Pullback{Tuple{typeof(Base.maybeconstructor), typeof(fvec1)}, Tuple{}}}, Zygote.var"#2041#back#221"{typeof(identity)}}}, Zygote.var"#2197#back#310"{Zygote.var"#308#309"{Tuple{Tuple{Nothing}, Tuple{Nothing}}}, Zygote.var"#2041#back#221"{typeof(identity)}}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), typeof(Zygote._jvec)}, Tuple{Zygote.Pullback{Tuple{typeof(Base.maybeconstructor), typeof(Zygote._jvec)}, Tuple{}}}, Zygote.var"#2041#back#221"{typeof(identity)}}}}}}})(Δ::Vector{Float64})

@ Zygote ~/.julia/packages/Zygote/Pm9ID/src/compiler/interface2.jl:0

[14] #308

@ ~/.julia/packages/Zygote/Pm9ID/src/lib/lib.jl:214 [inlined]

[15] #2197#back

@ ~/.julia/packages/ZygoteRules/CkVIK/src/adjoint.jl:72 [inlined]

[16] ComposedFunction

@ ./operators.jl:1050 [inlined]

[17] (::Zygote.Pullback{Tuple{ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, Vector{Float64}}, Tuple{Zygote.Pullback{Tuple{Type{NamedTuple}}, Tuple{}}}, Zygote.var"#2197#back#310"{Zygote.var"#308#309"{Tuple{Tuple{

```

```
{Nothing, Nothing}, Tuple{Nothing}}, Zygote.Pullback{Tuple{Base.var"##_113", @Kwargs{}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, Vector{Float64}}, Tuple{Zygote.Pullback{Tuple{typeof(Base.call_composed), Tuple{typeof(Zygote._jvec), typeof(fvec1)}, Tuple{Vector{Float64}}, @Kwargs{}}, Tuple{Zygote.var"#2057#back#230"{Zygote.var"#back#228"{2, 1, Zygote.Context{false}, typeof(Zygote._jvec)}}, Zygote.Pullback{Tuple{typeof(Zygote._jvec), Vector{Float64}}, Tuple{Zygote.Pullback{Tuple{typeof(vec), Vector{Float64}}, Tuple{}}}}, Zygote.var"#2169#back#298"{Zygote.var"#294#297"{Zygote.Pullback{Tuple{typeof(Base.call_composed), Tuple{typeof(fvec1)}, Tuple{Vector{Float64}}, @Kwargs{}}, Any}}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}}, Tuple{Zygote.var"#2208#back#320"{Zygote.var"#back#319"{:outer, Zygote.Context{false}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, typeof(Zygote._jvec)}}, Zygote.var"#2208#back#320"{Zygote.var"#back#319"{:inner, Zygote.Context{false}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, typeof(fvec1)}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), typeof(fvec1)}, Tuple{Zygote.Pullback{Tuple{typeof(Base.maybeconstructor), typeof(fvec1)}, Tuple{}}}, Zygote.var"#2041#back#221"{typeof(identity)}}}, Zygote.var"#2197#back#310"{Zygote.var"#308#309"{Tuple{Tuple{Nothing}, Tuple{Nothing}}, Zygote.var"#2041#back#221"{typeof(identity)}}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), typeof(Zygote._jvec)}, Tuple{Zygote.Pullback{Tuple{typeof(Base.maybeconstructor), typeof(Zygote._jvec)}, Tuple{}}}, Zygote.var"#2041#back#221"{typeof(identity)}}}}}}}, Zygote.var"#2041#back#221"{typeof(identity)}, Zygote.var"#2442#back#468"{Zygote.var"#pairs_namedtuple_pullback#467"{(), @NamedTuple{}}}}})(Δ::Vector{Float64})
```

@ Zygote ~/.julia/packages/Zygote/Pm9ID/src/compiler/interface2.jl:0

```
[18] (::Zygote.var"#91#92"{Zygote.Pullback{Tuple{ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, Vector{Float64}}, Tuple{Zygote.Pullback{Tuple{Type{NamedTuple}}, Tuple{}}}, Zygote.var"#2197#back#310"{Zygote.var"#308#309"{Tuple{Tuple{Nothing, Nothing}, Tuple{Nothing}}, Zygote.Pullback{Tuple{Base.var"##_113", @Kwargs{}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, Vector{Float64}}, Tuple{Zygote.Pullback{Tuple{typeof(Base.call_composed), Tuple{typeof(Zygote._jvec), typeof(fvec1)}, Tuple{Vector{Float64}}, @Kwargs{}}, Tuple{Zygote.var"#2057#back#230"{Zygote.var"#back#228"{2, 1, Zygote.Context{false}, typeof(Zygote._jvec)}}, Zygote.Pullback{Tuple{typeof(Zygote._jvec), Vector{Float64}}, Tuple{Zygote.Pullback{Tuple{typeof(vec), Vector{Float64}}, Tuple{}}}}, Zygote.var"#2169#back#298"{Zygote.var"#294#297"{Zygote.Pullback{Tuple{typeof(Base.call_composed), Tuple{typeof(fvec1)}, Tuple{Vector{Float64}}, @Kwargs{}}, Any}}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}}, Tuple{Zygote.var"#2208#back#320"{Zygote.var"#back#319"{:outer, Zygote.Context{false}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, typeof(Zygote._jvec)}}, Zygote.var"#2208#back#320"{Zygote.var"#back#319"{:inner, Zygote.Context{false}, ComposedFunction{typeof(Zygote._jvec), typeof(fvec1)}, typeof(fvec1)}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), typeof(fvec1)}, Tuple{Zygote.Pullback{Tuple{typeof(Base.maybeconstructor), typeof(fvec1)}, Tuple{}}}, Zygote.var"#2041#back#221"{typeof(identity)}}}, Zygote.var"#2197#back#310"{Zygote.var"#308#309"{Tuple{Tuple{Nothing}, Tuple{Nothing}}, Zygote.var"#2041#back#221"{typeof(identity)}}}, Zygote.Pullback{Tuple{typeof(Base.unwrap_composed), typeof(Zygote._jvec)}, Tuple{Zygote.Pullback{Tuple{typeof(Base.maybeconstructor), typeof(Zygote._jvec)}, Tuple{}}}, Zygote.var"#2041#back#221"{typeof(identity)}}}}}}}}}, Zygote.var"#2041#back#221"{typeof(identity)}, Zygote.var"#2442#back#468"{Zygote.var"#pairs_namedtuple_pullback#467"{(), @NamedTuple{}}}}})(Δ::Vector{Float64})
```

@ Zygote ~/.julia/packages/Zygote/Pm9ID/src/compiler/interface.jl:100

```
[19] withjacobian(f::Function, args::Vector{Float64})
      @ Zygote ~/.julia/packages/Zygote/Pm9ID/src/lib/grad.jl:190
[20] jacobian(f::Function, args::Vector{Float64})
      @ Zygote ~/.julia/packages/Zygote/Pm9ID/src/lib/grad.jl:168
[21] top-level scope
      @ ~/Library/CloudStorage/OneDrive-TheUniversityofMelbourne/JuliaPrograms/PekingUniGlobexCourse/Chapter04DifferentiationAndBoundaryValueProblems/jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_Y130sZmlsZQ==.jl:1
```

```
In [53]: Zygote.jacobian(fvec2,x)[1]
```

```
5×5 Matrix{Float64}:
 1.0  0.0  0.0  0.0  0.0
-4.0  1.0  0.0  0.0  0.0
 0.0 -4.0  1.0  0.0  0.0
 0.0  0.0 -4.0  1.0  0.0
 0.0  0.0  0.0 -1.0  1.0
```

```
In [54]: ForwardDiff.jacobian(fvec1,x)
```

```
5×5 Matrix{Float64}:
 1.0  0.0  0.0  0.0  0.0
-4.0  1.0  0.0  0.0  0.0
 0.0 -4.0  1.0  0.0  0.0
 0.0  0.0 -4.0  1.0  0.0
 0.0  0.0  0.0 -1.0  1.0
```

```
In [71]: ForwardDiff.jacobian(fvec2,x)
```

```
5×5 Matrix{Float64}:
 1.0  0.0  0.0  0.0  0.0
-4.0  1.0  0.0  0.0  0.0
 0.0 -4.0  1.0  0.0  0.0
 0.0  0.0 -4.0  1.0  0.0
 0.0  0.0  0.0 -1.0  1.0
```

The reason is because `Zygote` does not allow the array created inside the function to be mutated. In `fvec1()` the variable `df` is created inside the function and then mutated (changed). Because of the algorithm the `Zygote` uses, array mutation inside the function is not allowed.

Recall in the first week that we used `NonlinearSolve()` to find the solution to the two equations below

$$u(x, y) = x^2 - y + 1 = 0 \quad (1)$$

$$v(x, y) = 3 \cos(x) - y = 0 \quad (2)$$

We derived the Newton-Raphson formula

$$\begin{bmatrix} u_x & u_y \\ v_x & v_y \end{bmatrix} \begin{Bmatrix} x_{i+1} - x_i \\ y_{i+1} - y_i \end{Bmatrix} = - \begin{Bmatrix} u \\ v \end{Bmatrix}$$

In order to use the `ForwardDiff()`, it is more convenient to change the notation and write the above equations as

$$f_1(\vec{x}) = x_1^2 - x_2 + 1 = 0 \quad (3)$$

$$f_2(\vec{x}) = 3 \cos(x_1) - x_2 = 0 \quad (4)$$

$$\begin{bmatrix} \partial f_1 / \partial x_1 & \partial f_1 / \partial x_2 \\ \partial f_2 / \partial x_1 & \partial f_2 / \partial x_2 \end{bmatrix} \begin{Bmatrix} x_1^{(i+1)} - x_1^{(i)} \\ x_2^{(i+1)} - x_2^{(i)} \end{Bmatrix} = - \begin{Bmatrix} f_1(x_1^{(i)}, x_2^{(i)}) \\ f_2(x_1^{(i)}, x_2^{(i)}) \end{Bmatrix}$$

In the first week, we calculated the Jacobian by hand. Here we will use `ForwardDiff` to calculate the Jacobian

```
In [73]: VectorFunc(x)=[x[1]^2-x[2]+1,
                        3*cos(x[1])-x[2]]
```

VectorFunc (generic function with 1 method)

You can calculate the Jacobian by using `ForwardDiff.jacobian()`. In the example below, we evaluate the Jacobian at $(x_1, x_2) = (1, 1)$

```
In [74]: ForwardDiff.jacobian(VectorFunc, [1,1])
```

```
2x2 Matrix{Float64}:
 2.0      -1.0
-2.52441  -1.0
```

Now using Newton-Raphson formula

```
In [77]: x=[1,1]
for i=1:5
    F=VectorFunc(x)
    J=ForwardDiff.jacobian(VectorFunc,x)
    delta=-J\F
    x+=delta
    println(x)
end
```

```
[0.9162116530444182, 1.8324233060888364]
[0.9131363112988626, 1.8338084652856412]
[0.9131320007943952, 1.8338100508561948]
[0.913132000785916, 1.83381005085929]
[0.913132000785916, 1.83381005085929]
```